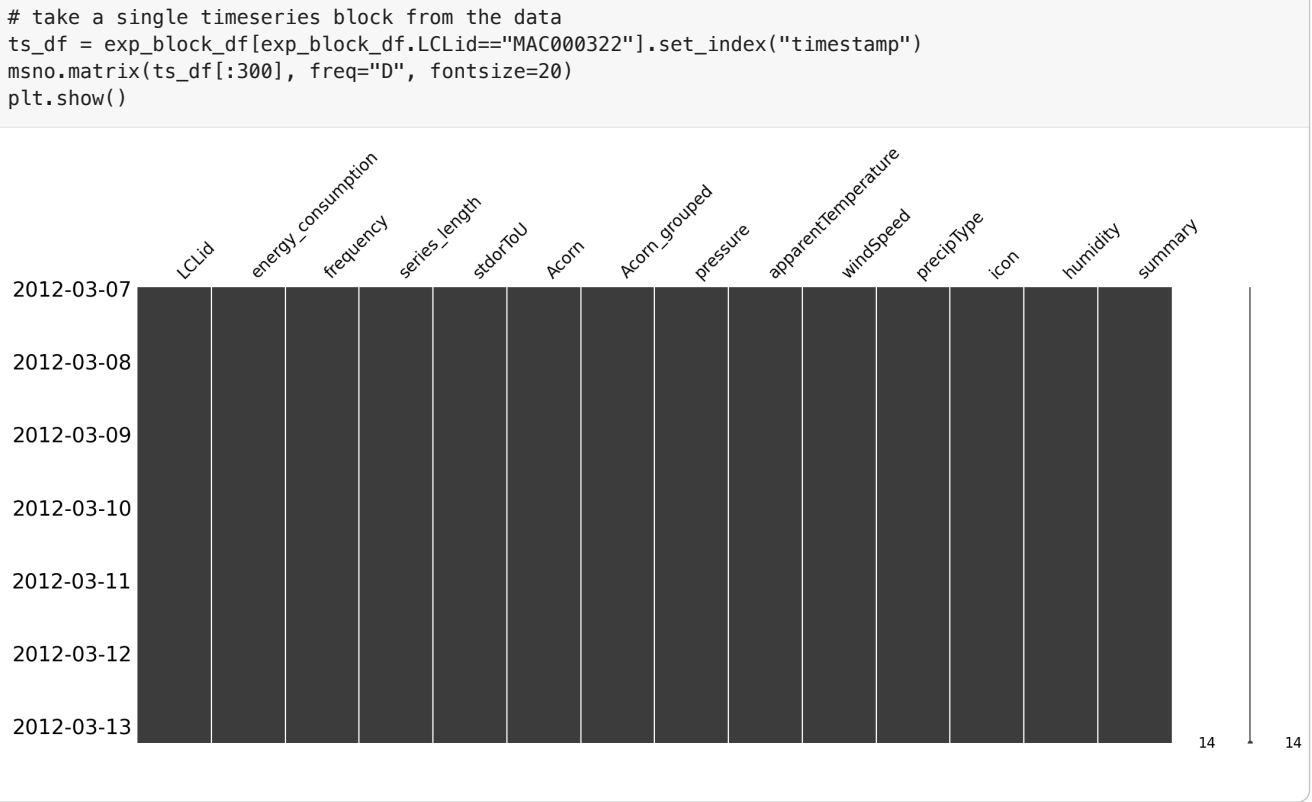
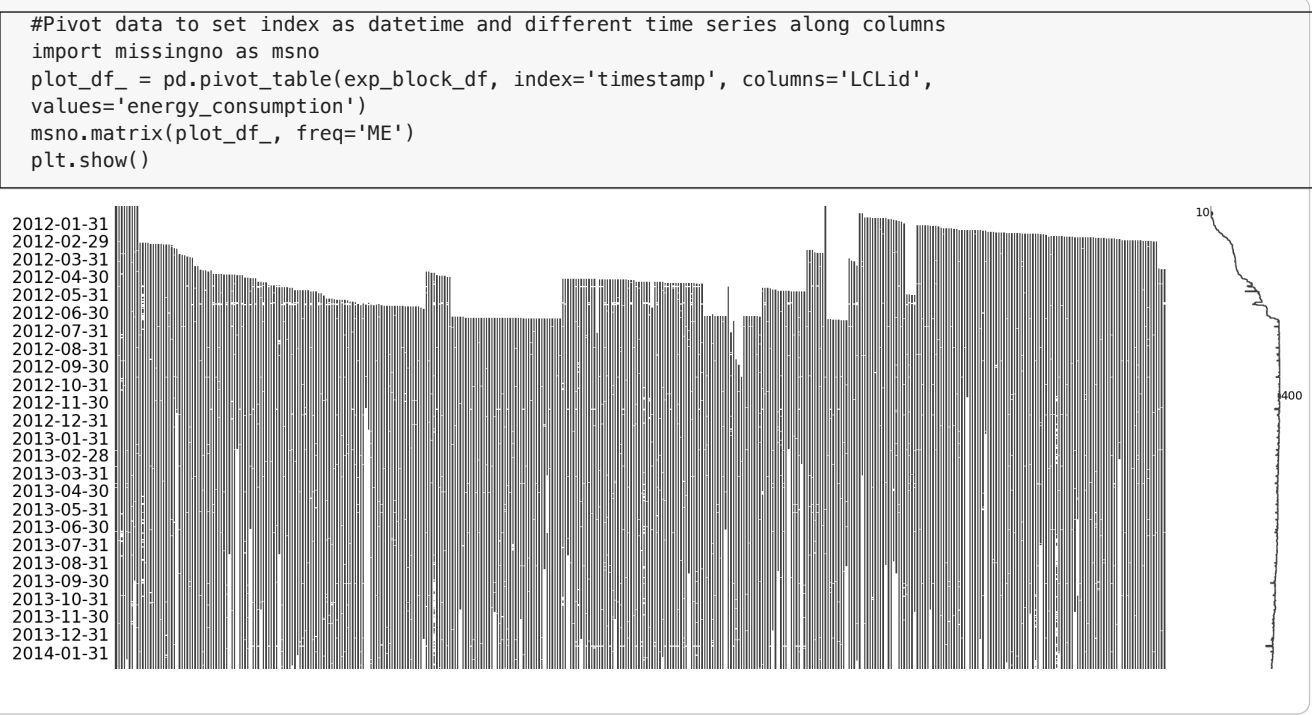


Preprocessing Time-Series Data

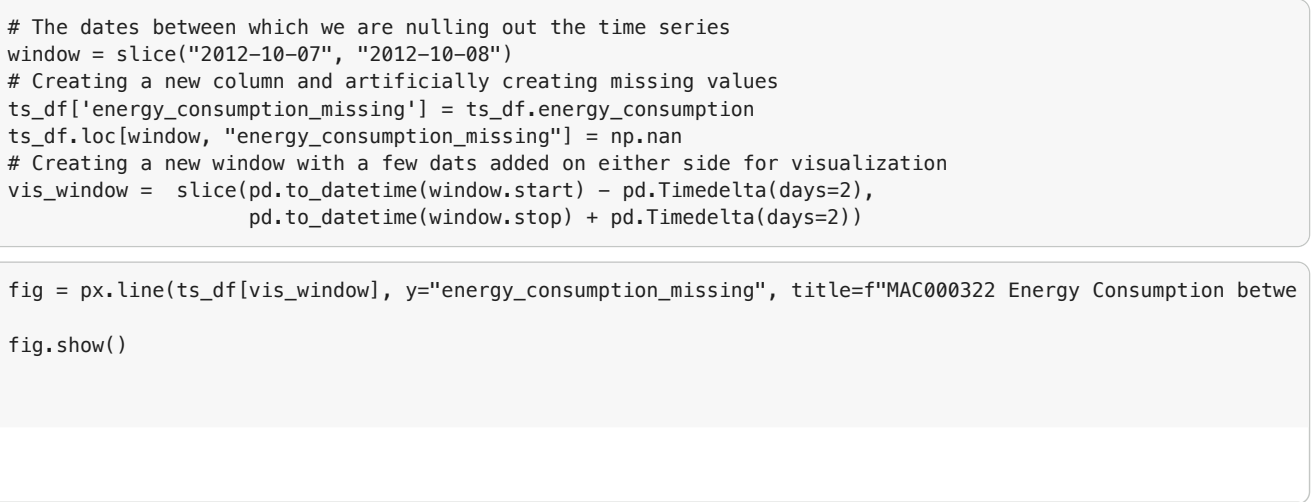
Visualizations

Baseline

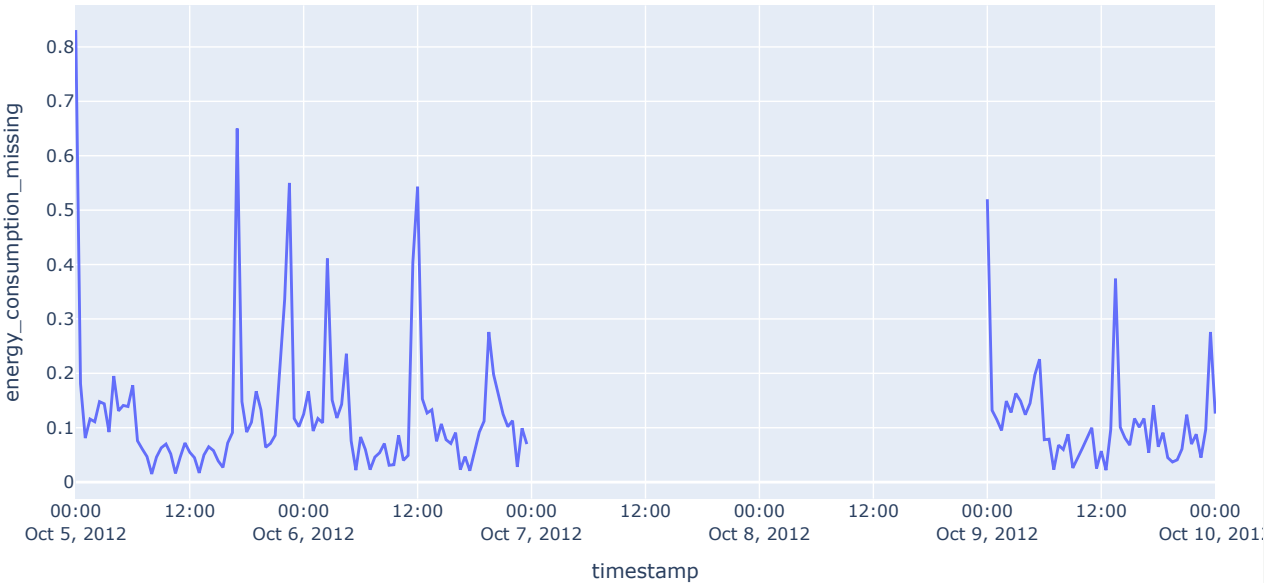
Missing Data and Imputation Techniques



Create a missing data section to impute the data and assess how the imputation technique works.

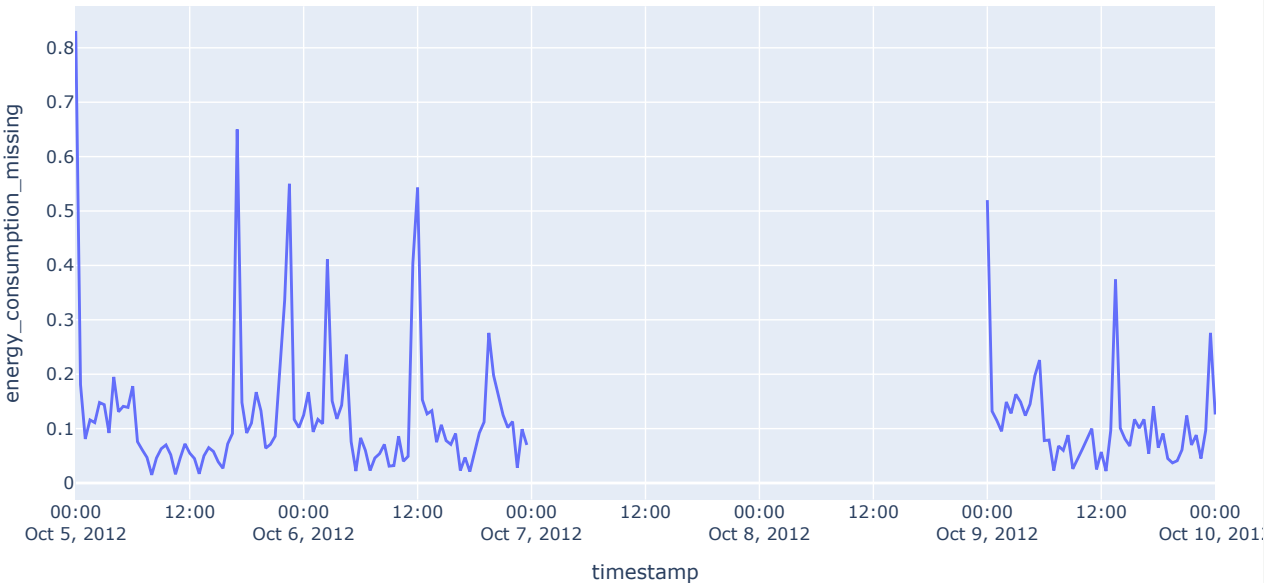


MAC000322 Energy Consumption between 2012-10-05 and 2012-10-10



```
fig = px.line(ts_df[vis_window], y="energy_consumption_missing", title=f"MAC000322 Energy Consumption between 2012-10-05 and 2012-10-10")
fig.show()
```

MAC000322 Energy Consumption between 2012-10-05 and 2012-10-10



Missing Data Imputation

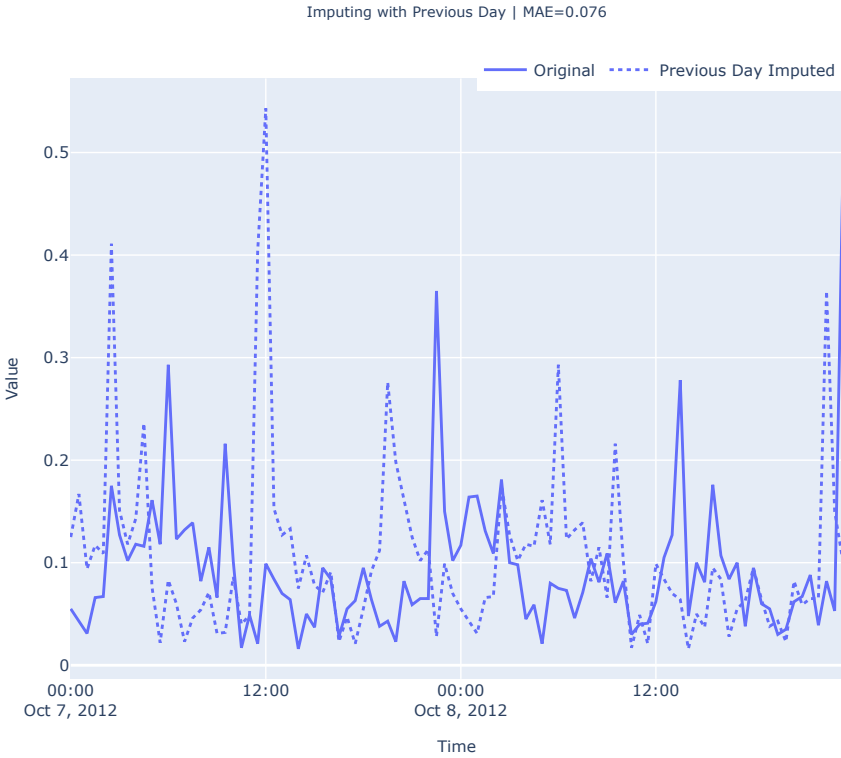
```
null_mask = ts_df.energy_consumption_missing.isnull()
```

Previous day

```
from sklearn.metrics import mean_absolute_error

ts_df['prev_day'] = ts_df['energy_consumption'].shift(48)
ts_df['prev_day_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc>null_mask, 'prev_day_imputed' = ts_df.loc>null_mask, 'prev_day'
mae = mean_absolute_error(ts_df.loc>window, 'prev_day_imputed', ts_df.loc>window, "energy_consumption"))

plot_df = pd.melt(ts_df.loc>window, ['energy_consumption', 'prev_day_imputed']).reset_index(), id_vars='times
fig = px.line(plot_df, x="timestamp", y=["value"], line_dash="series", title=f"Imputing with Previo
fig = format_plot(fig, ['Original', 'Previous Day Imputed'])
fig
```

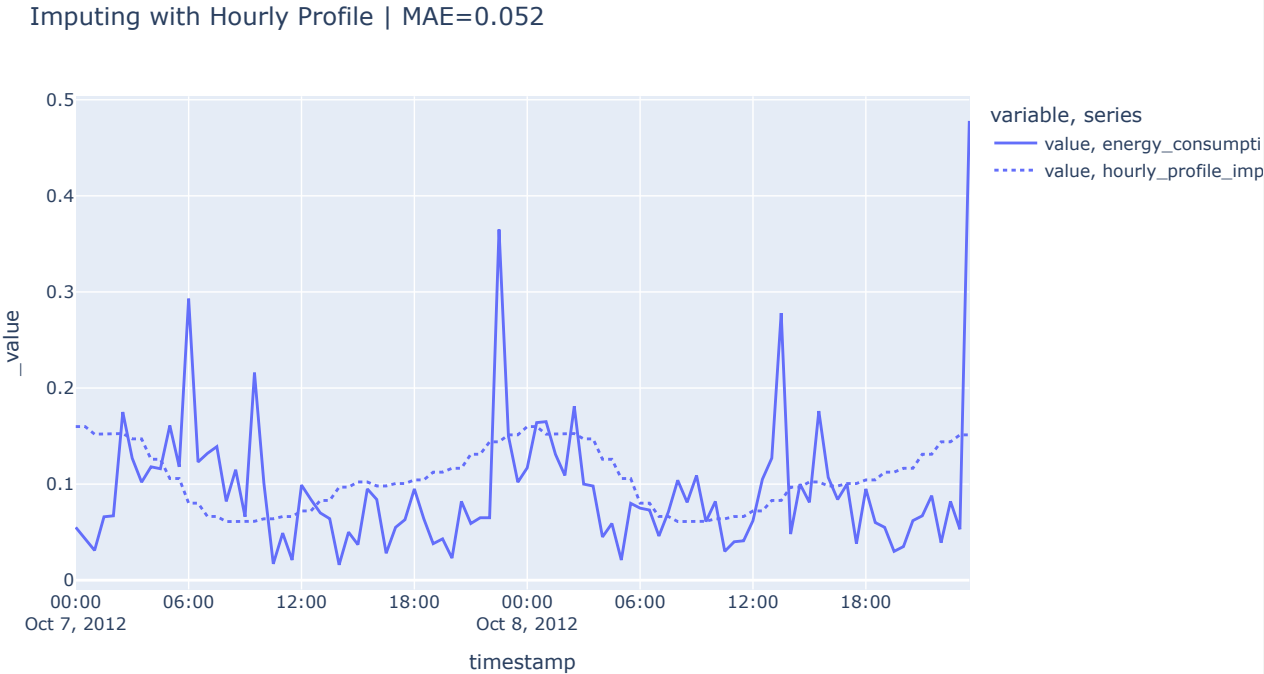


Hourly profile

```
ts_df["hour"] = ts_df.index.hour
hourly_profile = ts_df.groupby(['hour'])['energy_consumption'].mean().reset_index()
hourly_profile.rename(columns={'energy_consumption': 'hourly_profile'}, inplace=True)
idx = ts_df.index
ts_df = ts_df.merge(hourly_profile, on=['hour'], how='left', validate= 'many_to_one')
ts_df.index = idx

ts_df['hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask, 'hourly_profile_imputed'] = ts_df.loc[null_mask, 'hourly_profile']
mae = mean_absolute_error(ts_df.loc>window, 'hourly_profile_imputed', ts_df.loc>window, 'energy_consumption')
```

```
plot_df = pd.melt(ts_df.loc>window, ['energy_consumption', 'hourly_profile_imputed']).reset_index(), id_vars=
fig = px.line(plot_df, x="timestamp", y=["value"], line_dash="series", title=f'Imputing with Hourly Profile')
fig
```



Weekly hourly profile

```
ts_df["weekday"] = ts_df.index.weekday
day_hourly_profile = ts_df.groupby(['weekday', 'hour'])['energy_consumption'].mean().reset_index()
day_hourly_profile.rename(columns={"energy_consumption": "day_hourly_profile"}, inplace=True)
idx = ts_df.index

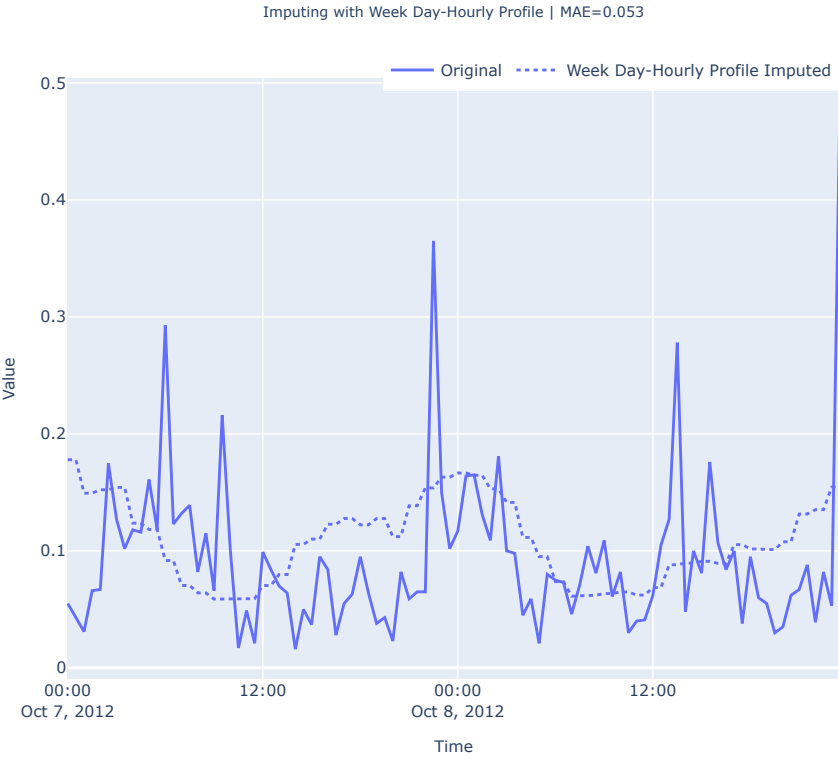
ts_df = ts_df.merge(day_hourly_profile, on=['weekday', 'hour'], how='left', validate='many_to_one')
ts_df.index = idx

#Using the day-hourly profile to fill missing
ts_df['day_hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask, "day_hourly_profile_imputed"] = ts_df.loc[null_mask, 'day_hourly_profile']
mae = mean_absolute_error(ts_df.loc>window, 'day_hourly_profile_imputed'], ts_df.loc>window, 'energy_consumption_missing')
```

```
plot_df = pd.melt(ts_df.loc>window, ['energy_consumption', 'day_hourly_profile_imputed']).reset_index(), id_

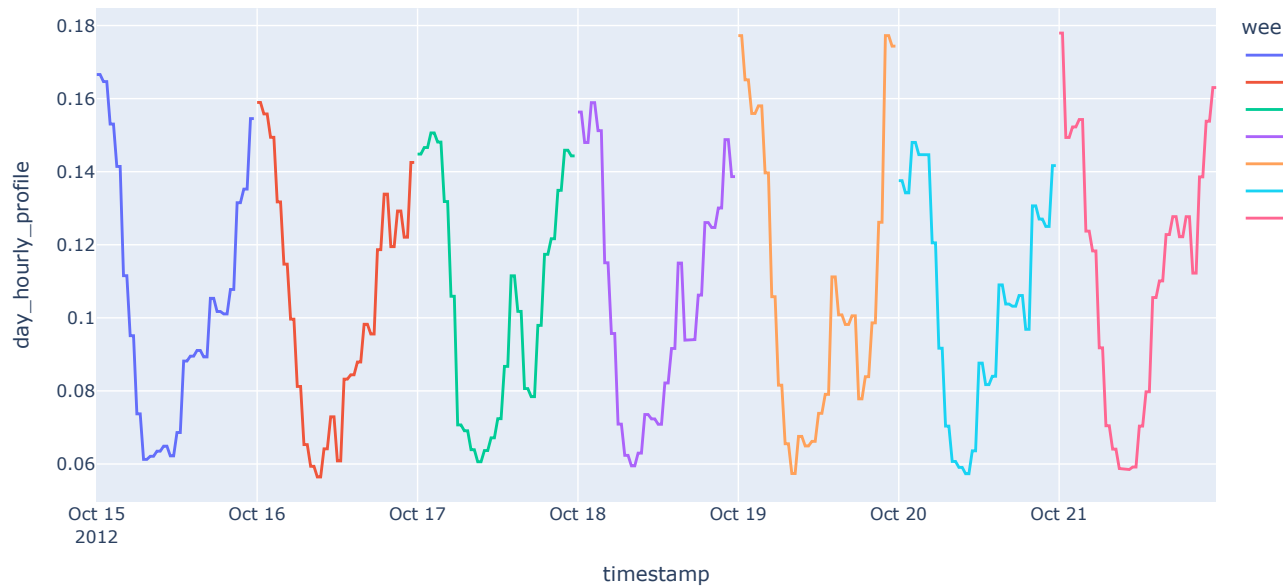
fig = px.line(plot_df, x="timestamp", y=["value"], line_dash="series", title=f'Imputing with Week Day-Hourl
fig = format_plot(fig, ['Original', 'Week Day-Hourly Profile Imputed'])

fig
```



```
px.line(ts_df.loc['2012-10-15': '2012-10-21'], ['day_hourly_profile', 'weekday']).reset_index(), x='timestamp'
```

Hourly Profiles by Weekday (Monday - Sunday)



Seasonal Interpolation

```
recovered_mx_iterp_weekday_half_hour = SeasonalInterpolation(
    seasonal_period=48*7,
    decomposition_strategy='additive',
    interpolation_strategy='spline',
    interpolation_args={'order':3},
    min_value=0).fit_transform(
    ts_df.energy_consumption_missing.values.reshape(-1,1))

ts_df['seas_iterp_weekday_half_hour_imputed'] = recovered_mx_iterp_weekday_half_hour

mae_weekday_half_hour = mean_absolute_error(
    ts_df.loc>window, 'seas_iterp_weekday_half_hour_imputed',
    ts_df.loc>window, 'energy_consumption')
```

```
recovered_mx_interp_half_hour = SeasonalInterpolation(
    seasonal_period=48,
    decomposition_strategy='additive',
    interpolation_strategy='spline',
    interpolation_args={'order':3},
    min_value=0).fit_transform(
    ts_df.energy_consumption_missing.values.reshape(-1,1))

ts_df['seas_interp_half_hour_imputed'] = recovered_mx_interp_half_hour

mae_half_hour = mean_absolute_error(
    ts_df.loc>window, 'seas_interp_half_hour_imputed',
    ts_df.loc>window, 'energy_consumption')
```

```
plot_df = pd.melt(ts_df.loc>window, ['energy_consumption',
    'seas_interp_half_hour_imputed', 'seas_iterp_weekday_half_hour_imputed']).
    reset_index(), id_vars='timestamp', var_name='series')

fig = px.line(plot_df, x="timestamp", y=["value"], line_dash="series",
    title=f'Imputing with Seasonal Interpolation <br> MAE Half Hourly=
    {mae_half_hour:.3f} | MAE Weekday-Half Hourly={mae_weekday_half_hour:.3f}')

fig = format_plot(fig, ['Original', 'Half Hourly Profile Imputed', 'Week
    Day-Half Hourly Profile Imputed'], title_font_size=16)

fig
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.